SOLID Principles in C#

Building Maintainable, Scalable, and Flexible Software

Presented By:

Alok Jaiswal

Date: 29-06-2026

- **S** Single Responsibility Principle
- **O** Open / Closed Principle
- **L** Liskov Substitution Principle
- **I** Interface Segregation Principle
- **D** Dependency Inversion Principle

Problems Before SOLID Principles

Tight Coupling

Classes depend directly on each other — one change breaks many.

Duplicate Code

Same logic copy-pasted in multiple places; hard to maintain.

Difficult Maintenance

A small fix requires touching dozens of unrelated files.

Difficult Testing

Tightly coupled code can't be unit tested in isolation.

Frequent Bugs

Any change in one module unexpectedly breaks another.

Poor Scalability

Adding features requires massive rewrites instead of extensions.

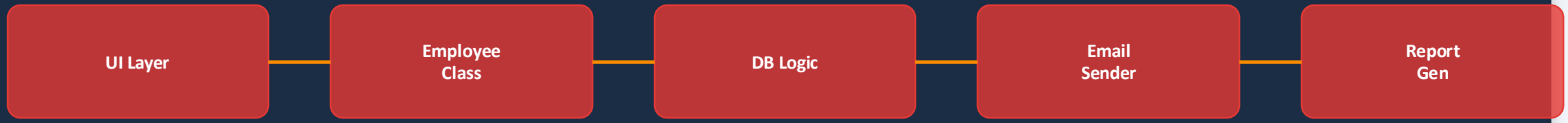
Low Reusability

Code written for one context can't be reused elsewhere.

Code Changes Affect Many

A single edit cascades through the entire codebase.

⚠ Messy & Tightly Coupled Architecture



All logic crammed into one class — changes ripple everywhere!

How SOLID Principles Solve These Problems

✗ BEFORE SOLID

- ✗ Tight coupling between classes
- ✗ Hard to modify without breaking
- ✗ Code duplication everywhere
- ✗ Difficult to unit test
- ✗ Cannot extend without editing
- ✗ Low code reusability
- ✗ Monolithic, rigid architecture

SOLID Principles

✓ AFTER SOLID

- ✓ Loose coupling — independent classes
- ✓ Easy to change without side effects
- ✓ High cohesion, no duplication
- ✓ Easily unit testable in isolation
- ✓ Open to extension, closed to modify
- ✓ High reusability across projects
- ✓ Scalable, modular architecture

Introduction to SOLID Principles

Robert C. Martin

"Uncle Bob"

American software engineer & author who introduced SOLID principles around 2000. Author of Clean Code and Clean Architecture.

What Can We Achieve?

- Clean Code
- Better Design
- Easy Maintenance
- Reusable Components
- Scalable Applications
- Easier Unit Testing

S — **Single Responsibility Principle**

One class = one job

O — **Open / Closed Principle**

Extend, don't modify

L — **Liskov Substitution Principle**

Subclass replaceable

I — **Interface Segregation Principle**

Small, focused interfaces

D — **Dependency Inversion Principle**

Depend on abstractions

Why SOLID Principles?

Reduced Dev Cost

Less time fixing bugs = lower cost per feature

Team Collaboration

Clean boundaries let teams work in parallel

Better Code Quality

Code that is readable, testable, and elegant

Easier Feature Addition

Extend with new classes, never break old ones

Enterprise Ready

Essential in .NET / C# enterprise architecture

SOLID principles are the foundation of professional C# / .NET development.
They are used in ASP.NET Core, Entity Framework, and all major enterprise architectures.

S Single Responsibility Principle (SRP)

"A class should have only one reason to change."

Real-Life Example

A Teacher teaches students.
An Accountant manages salaries.
Each has ONE responsibility.

Benefits of SRP

- Easy to maintain & debug
- Simple unit testing
- Reduced side effects
- Better code readability
- Promotes reusability

Before SRP

```
public class Employee
{
  public string Name { get; set; }

public decimal CalculateSalary()
 {
   return 50000m;
 }


public void GenerateReport()
 {
   Console.WriteLine("Report...");
 }


public void SaveToDatabase()
 {
   Console.WriteLine("Database...");
 }
}
```

After SRP

```
public class Employee
{
  public string Name { get; set; }
  public decimal Salary { get; set; }
}

public class SalaryService
{
 public decimal CalculateSalary (Employee e)
  {
    return e.Salary;
  }
}

public class ReportService
{
  public void GenerateReport (Employee e)
  {
    Console.WriteLine(e.Name);
  }
}

public class EmployeeRepository
{
  public void Save(Employee e)
  {
    // Save to DB
  }
}
```

O Open / Closed Principle (OCP)

> *"Software entities should be open for extension but closed for modification."*

🛒 Real-Life Example

Adding UPI / NetBanking to an online shop without changing existing payment code.

✅ Benefits of OCP

- No existing code modified
- New features = new classes
- Reduces regression bugs
- Supports plugin architecture

C# Code — OCP with Interface

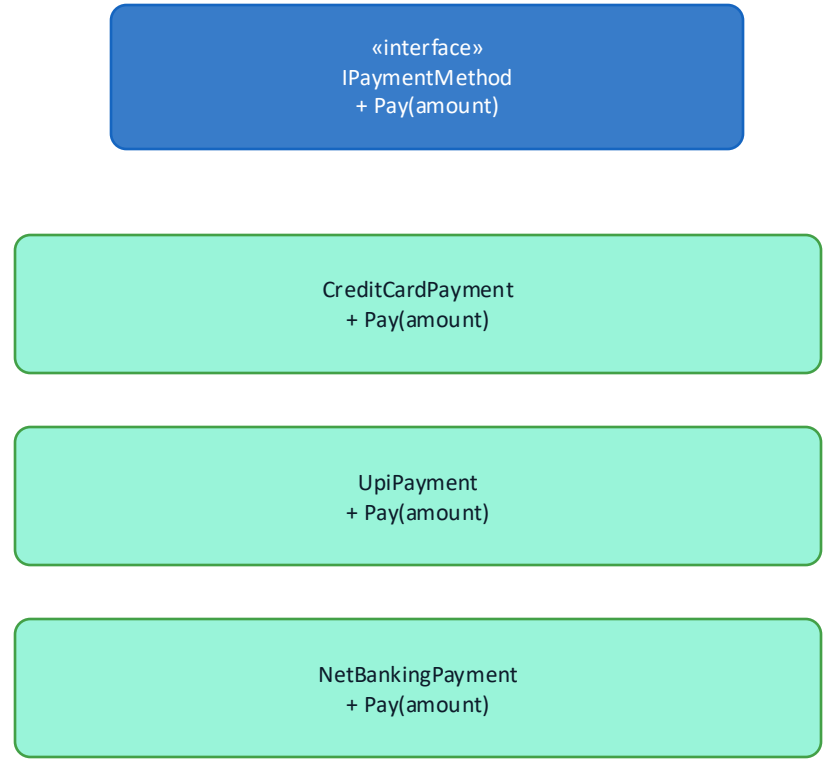
//Wrong Example:

```
class Payment
{
    public void Pay(string type)
    {
        if(type == "CreditCard")
        {
        }
        else if(type == "UPI")
        {
        }
    }
}
```

//Correct Example:

```
interface IPayment
{
    void Pay();
}

class CreditCardPayment :
IPayment
{
    public void Pay()
    {
    }
}

class UpiPayment : IPayment
{
    public void Pay()
    {
    }
}
```

UML Diagram



L Liskov Substitution Principle (LSP)

"Derived classes should be substitutable for their base classes without affecting correctness."

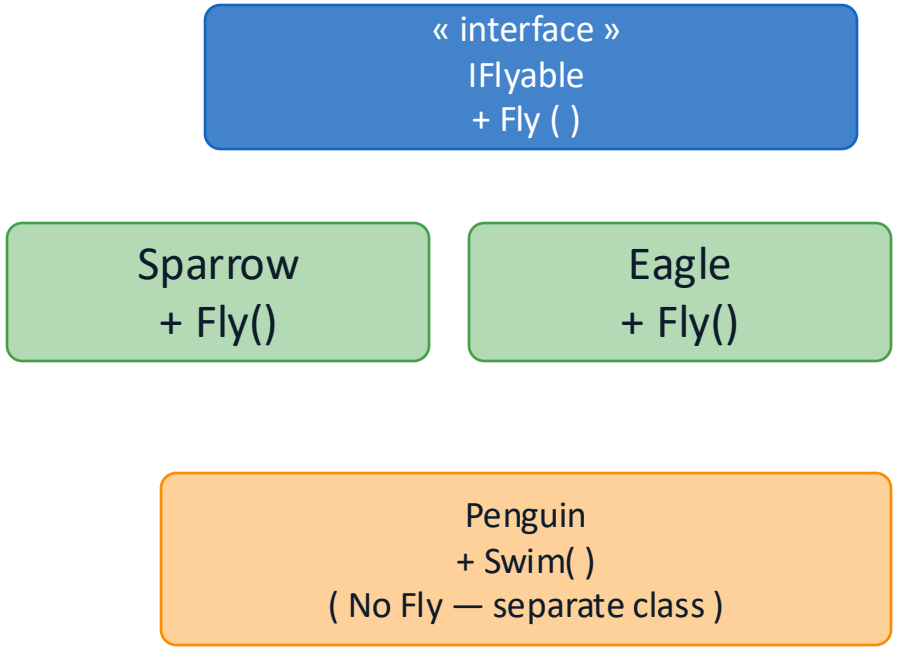
❌ **Wrong — Violates LSP**

```
class Bird
{
    public virtual void Fly()
    {
    }
}

class Ostrich : Bird
{
    public override void Fly()
    {
        throw new Exception();
    }
}
```

✅ **Correct — Follows LSP**

```
class Bird
{
}
interface IFly
{
    void Fly();
}

class Sparrow : Bird, IFly
{
    public void Fly()
    {
    }
}
class Ostrich : Bird
{
}
```

UML Diagram



Benefits: ✓ Safe polymorphism ✓ No unexpected exceptions ✓ Better abstractions ✓ Reliable class hierarchy

I Interface Segregation Principle (ISP)

"Clients should not be forced to depend on methods they do not use."

🍽️ Real-Life Example

A Restaurant Worker cooks food.
A Cashier processes payments.
They should not implement each other's methods.

Benefits

✓ Cleaner interfaces

✓ Less coupling

✓ Easier to implement

✓ Better testing

❌ Before ISP — Fat Interface

```
interface IWorker
{
    void Work();
    void Eat();
}

class Robot : IWorker
{
    public void Work()
    {
    }

    public void Eat()
    {
        throw new NotImplementedException();
    }
}
```

✅ After ISP — Segregated Interfaces

```
interface IWork
{
    void Work();
}

interface IEat
{
    void Eat();
}

class Human : IWork, IEat
{
    public void Work()
    {
    }

    public void Eat()
    {
    }
}

class Robot : IWork
{
    public void Work()
    {
    }
}
```

D Dependency Inversion Principle (DIP)

"High-level modules should not depend on low-level modules. Both should depend on abstractions."

Real-Life Example

A computer works with any USB keyboard.
The computer depends on a USB interface — not a specific keyboard brand.

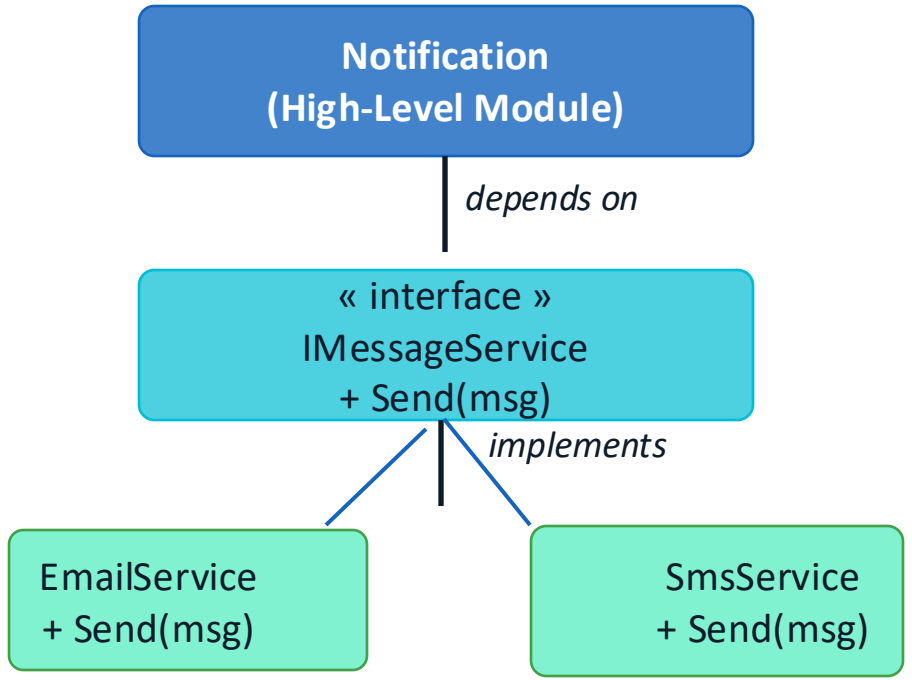
Benefits

✓ Swap implementations easily
✓ Easier unit testing with mocks
✓ Decoupled architecture
✓ Supports dependency injection

C# Code — Dependency Injection

```
//Wrong Example:

class EmailSender
{
public void Send()
{
Console.WriteLine("Email Sent");
}
}

class Notification
{
private EmailSender email = new
EmailSender();

public void Notify()
{
email.Send();
}
}
```

Dependency Injection Diagram



```
using System;

//step1:
public interface IMessage
{
void Send();
}
//step2:
public class EmailSender : IMessage
{
public void Send()
{
Console.WriteLine("Email Sent Successfully.");
}
}

// Step 3: Low-Level Module - SMS Sender
public class SmsSender : IMessage
{
public void Send()
{
Console.WriteLine("SMS Sent Successfully.");
}
}


// Step 4: High-Level Module
// Depends on the Interface, NOT on EmailSender or SmsSender
public class Notification
{
// Constructor Injection
public Notification(IMessage message)
{
this.message = message;
}

public void Notify()
{
message.Send();
}
}


// Step 5: Main Program
class Program
{
static void Main(string[] args)
{
// Send Email
IMessage email = new EmailSender();
Notification emailNotification = new Notification(email);
emailNotification.Notify();

Console.WriteLine();

// Send SMS
IMessage sms = new SmsSender();
Notification smsNotification = new Notification(sms);
smsNotification.Notify();
}
}
```

Conclusion

S Each class has a single responsibility — easier to maintain and debug.

O Extend behavior with new classes — never break existing code.

L Derived classes safely replace base classes — reliable hierarchy.

I Small, focused interfaces — no class implements unused methods.

D Depend on abstractions — swap implementations with zero impact.

"Clean code always looks like it was written by someone who cares." — Robert C. Martin

Presented by: Alok Jaiswal | MCA Final Year | GHRCEM Pune | 29-06-2026